Fatec
Registro

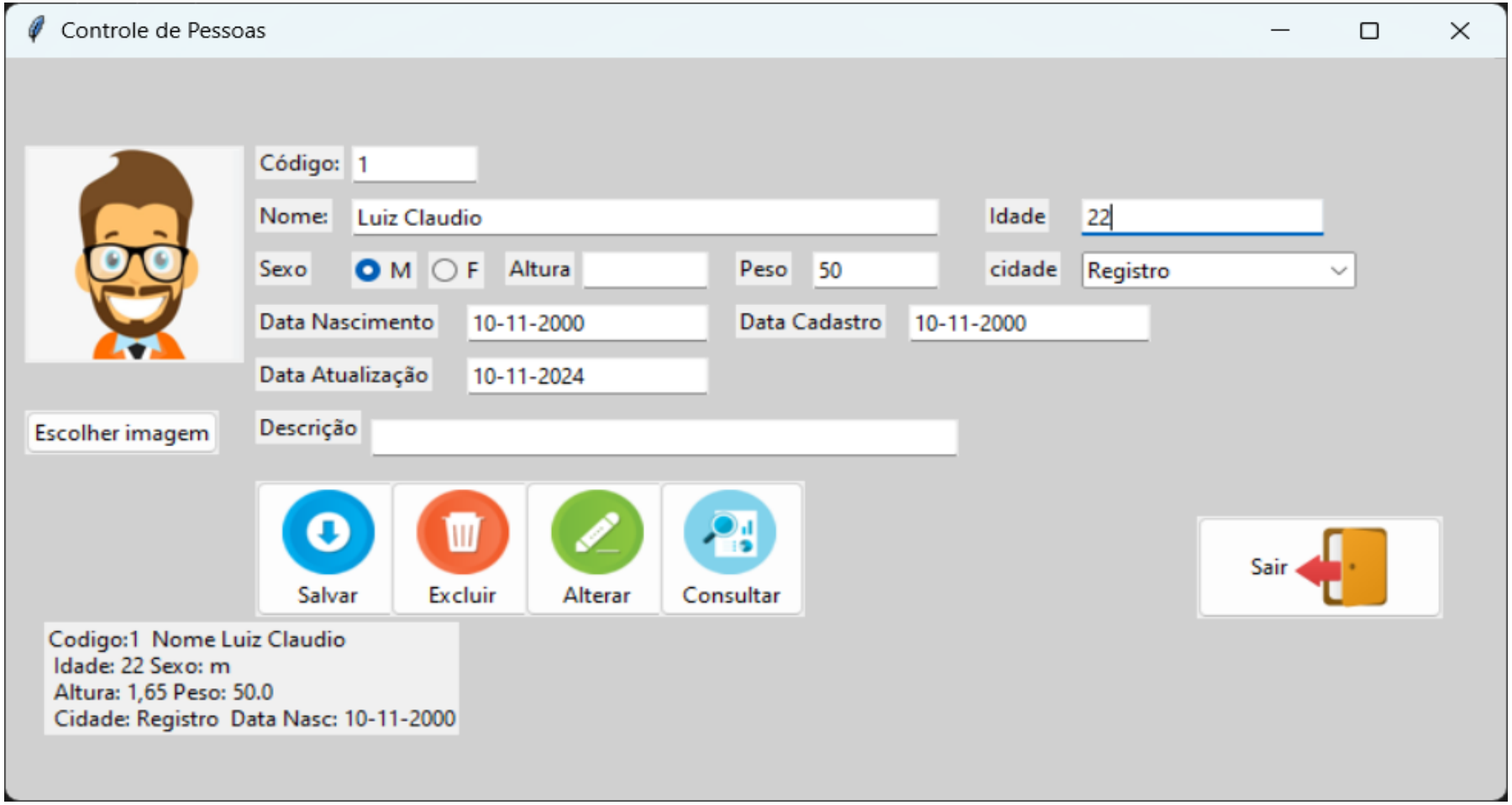


TÉCNICAS DE PROGRAMAÇÃO II

PROF° LUIZ CLÁUDIO

CRIANDO INTERFACE COM BANCO DE DADOS SQLITE

Depois de constuir uma interface para cadastro de Pessoas



CRIANDO BANCO DE DADOS SQLITE

```python
from tkinter import *
from tkinter import ttk, filedialog
from PIL import Image, ImageTk
import sys

#PARA IMPORTAR DADOS PARA O SQLITE
import sqlite3


class ControlePessoas:

    def __init__(self):
        self.tela = Tk()

        self.configurar_tela()
        self.criar_componentes()

        #CHAMA O MÉTODO PARA CRIAR O BANCO DE DADOS
        self.criar_banco()
```

Importe a biblioteca do sqlite

Crie um método para criar o banco de dados



CRIANDO BANCO DE DADOS SQLITE – MÉTODO CRIAR BANCO

```python
#CRIAR BANCO DE DADOS SQLITE
def criar_banco(self):
    self.conn = sqlite3.connect("pessoas.db")
    self.cursor = self.conn.cursor()

    self.cursor.execute("""
    CREATE TABLE IF NOT EXISTS pessoas (
        codigo INTEGER PRIMARY KEY,
        nome TEXT,
        idade INTEGER,
        sexo TEXT,
        altura REAL,
        peso REAL,
        cidade TEXT,
        data_nascimento TEXT,
        data_atualizacao TEXT,
        data_cadastro TEXT,
        descricao TEXT
    )
    """)
    self.conn.commit()
```

Define o nome do database

Cria a tabela Pessoas com os atributos



Fatec
Registro

CRIANDO BANCO DE DADOS SQLITE – MÉTODO SALVAR

```
def salvar(self):

#PARA SALVAR OS DADOS NO BD SQLITE
    try:
        self.cursor.execute("""
        INSERT INTO pessoas (
            codigo, nome, idade, sexo, altura, peso,
            cidade, data_nascimento, data_atualizacao,
            data_cadastro, descricao
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            self.txt_codigo.get(),
            self.txt_nome.get(),
            self.txt_idade.get(),
            self.var.get(),
            self.txt_altura.get(),
            self.txt_peso.get(),
            self.cmb_cidade.get(),
            self.txt_data_nascimento.get(),
            self.txt_data_atualizacao.get(),
            self.txt_data_cadastro.get(),
            self.txt_descricao.get()
        ))

        self.conn.commit()

        Label(self.tela, text="Dados salvos com sucesso!", fg="green").place(x=20, y=320)

    except Exception as e:
        Label(self.tela, text=f"Erro: {e}", fg="red").place(x=20, y=320)
```

Faz a inserção dos dados na tabela Pessoas

Pega os valores das caixas de texto



CRIANDO BANCO DE DADOS SQLITE – MÉTODO CONSULTAR

```python
#CONSULTAR DADOS SQLITE
def consultar_dados(self):
    for item in self.tree.get_children():
        self.tree.delete(item)

    self.cursor.execute("SELECT codigo, nome, idade, sexo, cidade FROM pessoas")
    for row in self.cursor.fetchall():
        self.tree.insert("", "end", values=row)
```

Faz a consulta dos dados na tabela Pessoas

```python
# --------------------------
def sair(self):
    self.tela.destroy()
    #FECHA A CONEXÃO COM SQLITE
    self.conn.close()
    self.tela.destroy()
    sys.exit()
```

Fecha a classe de conexão ao sair



CRIANDO BANCO DE DADOS SQLITE – CRIA TABELA PARA MOSTRAR DADOS

```
#TABELA PARA MOSTRAR DADOS SQLITE
self.tree = ttk.Treeview(self.tela, columns=("cod","nome","idade","sexo","cidade"), show="headings")

self.tree.heading("cod", text="Código")
self.tree.heading("nome", text="Nome")
self.tree.heading("idade", text="Idade")
self.tree.heading("sexo", text="Sexo")
self.tree.heading("cidade", text="Cidade")

self.tree.place(x=20, y=350, width=750, height=200)

self.tree.bind("<ButtonRelease-1>", self.selecionar_item)
```

Colunas que serão usadas na tabela

Itens que irão aparecer na tabela



CRIANDO BANCO DE DADOS SQLITE – SELECIONAR ITENS DA TABELA

```python
#SELECIONAR ITEMS DA TABELA BANCO DE DADOS
def selecionar_item(self, event):
    item = self.tree.selection()[0]
    dados = self.tree.item(item, "values")

    self.txt_codigo.delete(0, END)
    self.txt_codigo.insert(0, dados[0])

    self.txt_nome.delete(0, END)
    self.txt_nome.insert(0, dados[1])

    self.txt_idade.delete(0, END)
    self.txt_idade.insert(0, dados[2])

    self.var.set(dados[3])
    self.cmb_cidade.set(dados[4])
```

Seleciona os itens da tabela e mostra nos componentes



CRIANDO BANCO DE DADOS SQLITE – ATUALIZAR DADOS TABELA

```
#ATUALIZAR DADOS SQLITE
def atualizar(self):
    self.cursor.execute("""
    UPDATE pessoas SET
        nome=?,
        idade=?,
        sexo=?,
        altura=?,
        peso=?,
        cidade=?,
        data_nascimento=?,
        descricao=?
    WHERE codigo=?
    """, (
        self.txt_nome.get(),
        self.txt_idade.get(),
        self.var.get(),
        self.txt_altura.get(),
        self.txt_peso.get(),
        self.cmb_cidade.get(),
        self.txt_data_nascimento.get(),
        self.txt_descricao.get(),
        self.txt_codigo.get()
    ))

    self.conn.commit()
    self.consultar_dados()
```

Pega os valores dos componentes e atualiza na tabelas pessoas



CRIANDO BANCO DE DADOS SQLITE – EXCLUIR DADOS TABELA

```
#EXCLUIR DADOS BANCO DE DADOS
def excluir(self):
    self.cursor.execute("DELETE FROM pessoas WHERE codigo=?", (self.txt_codigo.get(),))
    self.conn.commit()
    self.consultar_dados()
    Label(self.tela, text="Dados excluidos com sucesso!", fg="green").place(x=20, y=320)
```

Exclui os dados da tabela pessoas , pelo atributo código





OBRIGADO

LUIZ.BARRETO@CPS.SP.GOV.BR